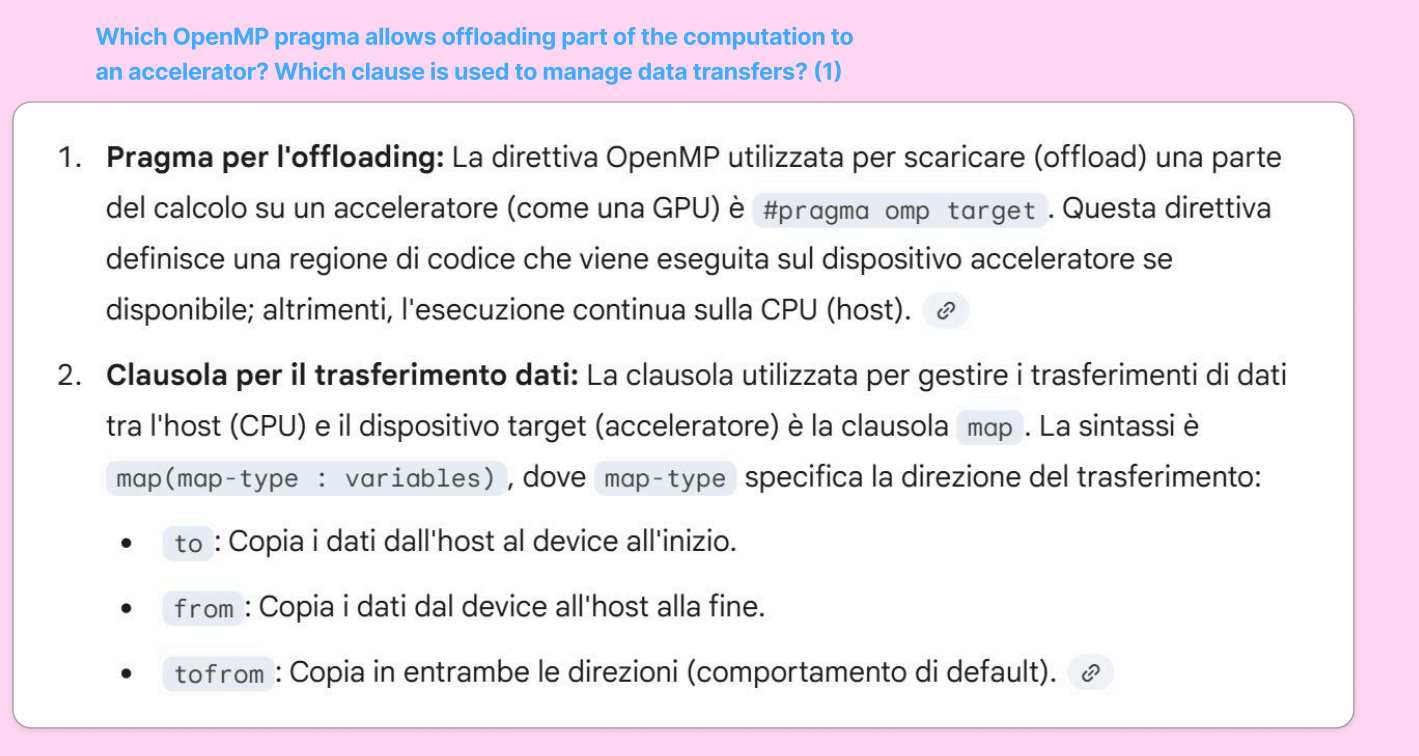
OpenMP

OpenMP

- **definition**
  - the idea is to raise the level of abstraction
    - instead of writing low-level code to create and join threads, you simply tell the compiler "*this section should be parallel*"
  - **OpenMP** (Open Multi-Processing) is an API for multi-threaded shared memory programming
    - it relies heavily on compiler support
      - perché stiamo scrivendo su C++
  - it consists of three parts
    - **compiler directives:** this is about 80% of what you use
      - you annotate your code
    - **library routines:** about 19% of the usage
      - e.g. "*how many threads am I running?*"
    - **environment variables:** setting runtime parameters from the shell
  - to use it in C/C++
    - we include amp.h
    - compile with the -fopenmp flag
- **why use OpenMP over Pthread?**
  - **portability**
    - it is standard across various architectures
      - the same code works on your laptop and a supercomputer
      - standard across a variety of shared memory architectures and platforms
      - scalable from embedded systems to the supercomputer
    - it supports Fortran, C and C++
  - **ease of use**
    - you can take a serial program and apply incremental parallelization
      - we don't have to rewrite the whole program
      - we just add directives to the slow loops
        - 3 or 4 are enough to get significant speedups
- **the architectural model: fork-join**
  - OpenMP uses a specific execution model called **Fork-Join** that has this lifecycle
    - **master thread:** your program starts as a single process with one thread, the "Master"
    - **fork:** when the master reaches a parallel region, it "forks" (spawns) a team of **slave threads**
    - **parallel execution:** the task is divided among these slaves and they run concurrently on different processors
    - **join:** at the end of the parallel region, the thread synchronize (join)
      - the slaves die or suspend
      - the master continues alone until the next parallel region
  - this implies a hierarchy where the Master manages the flow and synchronization happens at the "Join" point

Which OpenMP pragma allows offloading part of the computation to an accelerator? Which clause is used to manage data transfers? (1)



1. **Pragma per l'offloading:** La direttiva OpenMP utilizzata per scaricare (offload) una parte del calcolo su un acceleratore (come una GPU) è `#pragma omp target`. Questa direttiva definisce una regione di codice che viene eseguita sul dispositivo acceleratore se disponibile; altrimenti, l'esecuzione continua sulla CPU (host).
2. **Clausola per il trasferimento dati:** La clausola utilizzata per gestire i trasferimenti di dati tra l'host (CPU) e il dispositivo target (acceleratore) è la clausola `map`. La sintassi è `map(map-type : variables)`, dove `map-type` specifica la direzione del trasferimento:
   - `to`: Copia i dati dall'host al device all'inizio.
   - `from`: Copia i dati dal device all'host alla fine.
   - `tofrom`: Copia in entrambe le direzioni (comportamento di default).

- **the OpenMP Language Structure**
  - the API is structured into five main categories
    - **parallel control structures**
      - these goven the flow control
      - e.g., parallel directive
    - **work sharing**
      - how do we distribute the work?
      - e.g., splitting a for loop among threads
    - **data environment**
      - who sees what variable?
      - defining shared vs private scopes
    - **synchronization**
      - coordinating threads
      - e.g., critical sections, barriers
    - **runtime functions**
      - querying the system
      - e.g., omp_get_thread_num()
- **basic syntax: the pragma**
  - **pragma** (pragmatic information) are the main part of the OpenMP standard
    - identify tasks and their synchronizations
    - `#pragma omp <name> [list of clauses]`
      - ex. #pragma omp parallel tells the compiler that the following block is a parallel region
        - if we remove these pragmas, the program should still compile and run correctly as a serial program
  - the auxiliary C functions
    - used to set and get relevant informations such as number of available threads
    - used to manage explicit locks
- **parallel control structures**
  - to initiate any parallelism, we use the directive #pragma omp parallel

    ```
    #pragma omp parallel
      {/* parallel section */}
    ```

    - when the master thread encounters this directive, it spawns a team of threads
      - this is the "Fork" in the Fork-Join model
      - the thread that was executing the code becomes the master (thread ID 0)
    - by default the code inside the {...} block is **replicated**
      - if you have 4 threads, every single thread executes the entire block of code
    - at the end of the block there is an implies barrier and only the master thread continues
  - we can add optional clauses to the parallel directive
    - conditional parallelization with if(condition)
      - it parallelize only if a specific condition is true (e.g., "only parallelize if N>1000")
    - number of spawned thread with num_threads(int)
      - explicitly ask for a specific number of threads
  - under the hood, your compiler might replace the directive with a Pthreads implementation

    ```
    #pragma omp parallel num_threads(8)
    for (i = 0; i < 8; i++)
      pthread_create(...);
    for (i = 0; i < 8; i++)
      pthread_join(...);
    ```

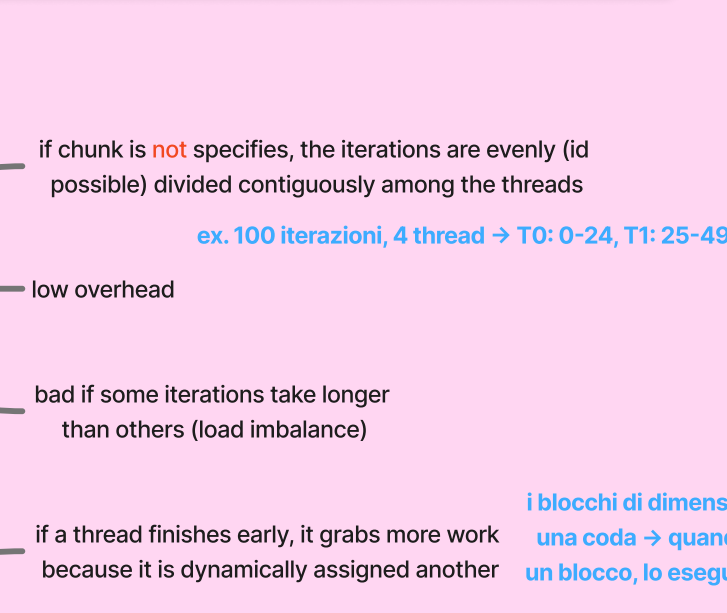
  - the system decide how many threads to give you with a strict precedence order
    - evaluation of the if clause
      - scriviamo una condizione booleana direttamente nella direttiva come #pragma omp parallel if(condizione) → quando N è piccolo, OpenMP ignora qualsiasi richiesta di thread multipli ed esegue un codice seriale
    - the num_threads clause
    - the omp_set_num_threads() library function
      - funzione di C++ chiamata prima della regione parallela che imposta una variabile interna globale dentro il programma
    - the OMP_NUM_THREADS environment variable
      - è una configurazione data al sistema operativo prima di lanciare il programma, scritta nel terminale
    - the system default (usually the number of CPU cores)
      - configurazione hardware di base
- **work sharing constructs**
  - executing the exact same code on 4 threads (replication) isn't always useful
    - usually, we want to split a big task into 4 smaller pieces
      - **work sharing**
  - a work sharing construct divided the execution of a code region among the members of the team
    - work sharing constructs do not launch new threads but they use the existing team created by #pragma omp parallel
      - a work sharing construct must be *enclosed within* a parallel region in order for the directive to execute in parallel
    - there is usually an implied barrier at the end of a work-sharing construct but no barrier at the entry
      - unless we use nowait for the barrier at the end
  - four main ways to share work
    - **for construct**

      ```
      #pragma omp parallel
      {
        #pragma omp for
        /*for loop*/
      }
      ```

      affinché funzioni il ciclo deve essere in forma canonica → OpenMP deve poter calcolare il numero esatto di iterazioni prima che il ciclo inizi

      - **#pragma omp for** splits the iterations of a for loop across the threads
        - if you have 100 iterations and 4 threads, OpenMP might give 0-24 to Thread 1, 25-49 to Thread 2 ans o on
        - it is used for **data parallelism**

      altre clausole

      `nowait`

      Di default, alla fine di un `#pragma omp for` c'è una **barriera implicita**. Tutti i thread aspettano che l'ultimo abbia finito il suo pezzo di ciclo prima di proseguire.

      - Se aggiungi `nowait`, i thread che finiscono prima proseguono subito con il codice successivo (se possibile).

      `collapse(n)`

      Utile per i cicli annidati.

      - Se hai due cicli `for` uno dentro l'altro (es. matrice i, j), normalmente OpenMP parallelizza solo quello esterno (`i`).
      - Con `collapse(2)`, OpenMP unisce i due cicli in un unico "mega-ciclo" lineare e parallelizza quello. Aumenta il numero di iterazioni disponibili, migliorando il parallelismo se il ciclo esterno è troppo breve (es. meno iterazioni del numero di thread).

      `ordered`

      Impone che una certa parte del ciclo (quella racchiusa in `#pragma omp ordered`) venga eseguita nello stesso ordine che avrebbe avuto in un ciclo seriale (es. 0, 1, 2...). Uccide le performance, si usa solo se strettamente necessario (es. per stampe a video ordinate).

      - **scheduling**

        clausola come schedule(tipo, chunk_size)

        - how does it decide which thread gets which iteration? via the schedule clause
          - **static**: iterations are divided into fixed-size chunks and assigned at the start

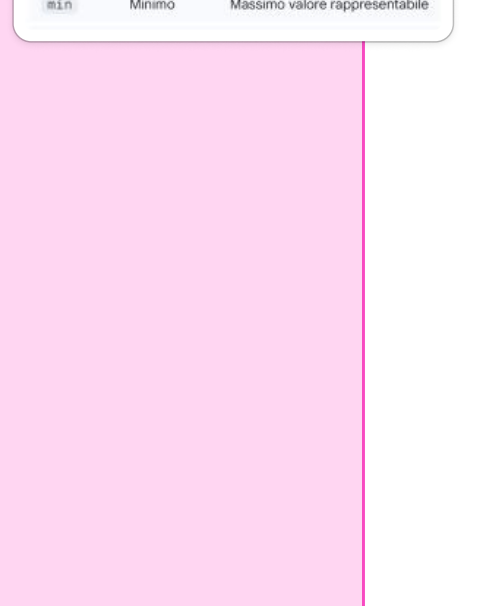
            si usa se ogni iterazione richiede lo stesso tempo di esecuzione

            - if chunk is not specifies, the iterations are evenly (id possible) divided contiguously among the threads
              - ex. 100 iterazioni, 4 thread → T0: 0-24, T1: 25-49, etc
            - low overhead
            - bad if some iterations take longer than others (load imbalance)
          - **dynamic**: iterations are handed out in chunks as threads become availabe

            si usa quando il carico di lavoro è sbilanciato

            - if a thread finishes early, it grabs more work because it is dynamically assigned another
              - i blocchi di dimensione chunk vengono messi in una coda → quando un thread è libero, prende un blocco, lo esegue e torna a chiederne un altro
            - the default chunk size is 1
            - higher overhead but excellent load balancing
          - **guided:** similar to dynamic, but the chunk size starts large and gets smaller trying to balance overhead and load balancing
            - i blocchi iniziano grandi (per ridurre l'overhead) e diventano esponenzialmente più piccoli verso la fine del ciclo (per bilanciare il carico)

            si usa se si vogliono i benefici del dinamico ma si vogliono ridurre il numero di volte che i thread devono accedere alla coda dei lavori
      - **handling sum: reduction**
        - if we are calculating a sum inside a parallel loop there is the risk that a data race happens
        - to solve this easily, OpenMP provides the reduction(operator: list) clause
          - it creates a privare copy of the variable for each thread
            - ogni copia viene inizializzata al valore neutro (identità) dell'operatore che hai scelto, ignorando il valore originale della variabile globale per il momento
          - each thread updates its local copy
          - at the end, all copies are combined (reduced) using the specified operator (like + or *) into the global variable

Operatore	Tipo	Valore Iniziale (Identità)
`+`	Somma	`0`
`*`	Prodotto	`1`
`-`	Sottrazione	`0`
`&`	Bitwise AND	`~0` (tutti 1)
` `	` `	Bitwise OR
`^`	Bitwise XOR	`0`
`&&`	Logical AND	`1` (True)
` `		` `
`max`	Massimo	Minimo valore rappresentabile
`min`	Minimo	Massimo valore rappresentabile

    - **sections construct**
      - while loops split *data*, **Sections** split *code*

        all'interno di un blocco #pragma omp sections, definisci diversi blocchi #pragma omp section → ogni "section" viene presa in carico ed eseguita da un thread diverso del team

        ```
        #pragma omp parallel
        {
          #pragma omp sections
          {
            #pragma omp section
            {
              /*code section 1*/
            }
            #pragma omp section
            {
              /*code section 2*/
            }
          }
        }
        ```

        barriera implicita alla fine di ogni sections

        - specifies that the enclosed section(s) of code are to be executed in parallel
        - each section is executed once by a thread in the team
      - if you have two completely different tasks - Task A and Task B - we can run them in parallel
    - **single and master**
      - sometimes, inside a parallel ragion, you need a specific action to happen only once
        - like printing a result or initializing a variable

        ```
        #pragma omp parallel
        {
          #pragma omp single
          {
              /*code section*/
          }
          #pragma omp master
          {
              /*code section*/
          }
        }
        ```

        - **#pragma omp single**
          - only one thread executes this block (it could be any thread)
            - il primo thread che arriva al blocco esegue l'azione mentre tutti gli altri aspettano alla fine del blocco single (barriera implicita)
          - everyone else waits at the end (unless nowait is used)
        - **#pragma omp master**
          - only the master thread (Thread 0) executes this
          - there is no implicit barrier at the end of the master block so other threads keep going immediately
            - più veloce di single perché non c'è l'attesa
    - **tasks**
      - for irregular problems (like traversing a linked list or recursion) where you can't calculate loop bound easily, OpenMP 3.0 introduced **tasks**

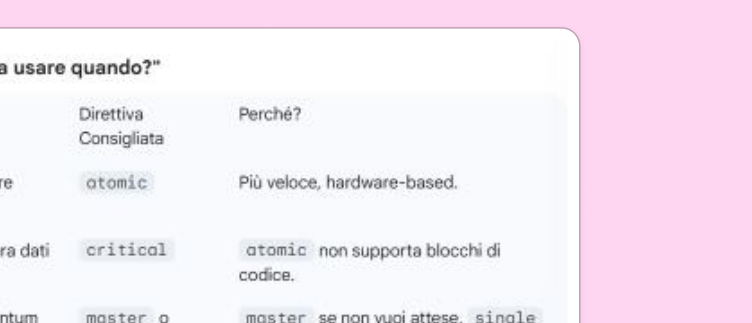
        serve per gestire il parallelismo irregolare, come i cicli while o la visita di strutture dati complesse, dove non sai a priori quante iterazioni farai (ad esempio se abbiamo un albero non sappiamo quanti nodi ci sono finché non arriviamo in fondo)

        ```
        #pragma omp task
        {
          /*code section*/
        }
        #pragma omp taskwait
        #pragma omp taskyield
        ```

        - **#pragma omp task**
          - we create a unit of work that is added to a queue
          - threads in the team pull tasks from this queue and execute them
            1. abbiamo che un solo thread si occupa di scorrere la lista
            2. crea un post-it con scritto "elabora questo nodo" (#pragma omp task)
            3. questo sarà eseguito da un thread libero (chiunque)
        - **taskwait**
          - acts as a barrier, ensuring all sibling tasks are complete before proceeding
        - **taskyield**
          - directive interrupts execution of the current task
          - may be resumed by the same thread (tied clause) or by another (untied)
- **synchronization directives**
  - we need explicit synchronization when threads need to access the same memory or coordinate their timing
    - **critical directive**

      ```
      #pragma omp critical [name]
      {
        /*code section*/
      }
      ```

      - we use it when we have a block of code that must be executed by only one thread at a time
      - when a thread reaches this, it checks if anyone else is executing it
        - if so, it waits
      - **crucial detail**
        - if we have two critical sections protecting *different* data, we have to give them different names
        - if they are unnamed, they map to the same unspecified name, meaning a thread updating variable A might unnecessarily block a thread updating variable B

      Dettaglio importante (Nomi): Puoi dare un nome alla sezione critica (es. `#pragma omp critical (saldo)`).

      - Se due sezioni critiche hanno lo *stesso nome*, sono trattate come lo stesso semaforo (bloccanti tra loro).
      - Se hanno *nomi diversi*, due thread possono eseguirle contemporaneamente (non si bloccano a vicenda).
      - Se non metti il nome, tutte le sezioni critiche senza nome del programma si bloccano a vicenda.

    - **barrier directive**
      - `#pragma omp barrier`

        sincronizza esplicitamente tutti i thread del team

        - when a thread reaches a barrier, it waits until *all* other threads in the team arrive
          - we use when we need the entire team to wait until everyone reaches a specific point
        - **note**
          - OpenMP often puts implicit barriers at the end of work-sharing constructs (like for or single), so explicit barriers are less common than in Pthreads
    - **atomic directive**
      - `#pragma omp atomic`
        - this ensures that a specific storage location is accessed atomically
          - no multiple simultaneous writes
        - faster than a critical section
          - sfrutta le istruzioni hardware specifiche della CPU ma non richiede il costo elevato di gestione di un lock software come critical
        - only applies to the single statement immediately following the directive
          - ex. non funziona con printf ma con x+=y

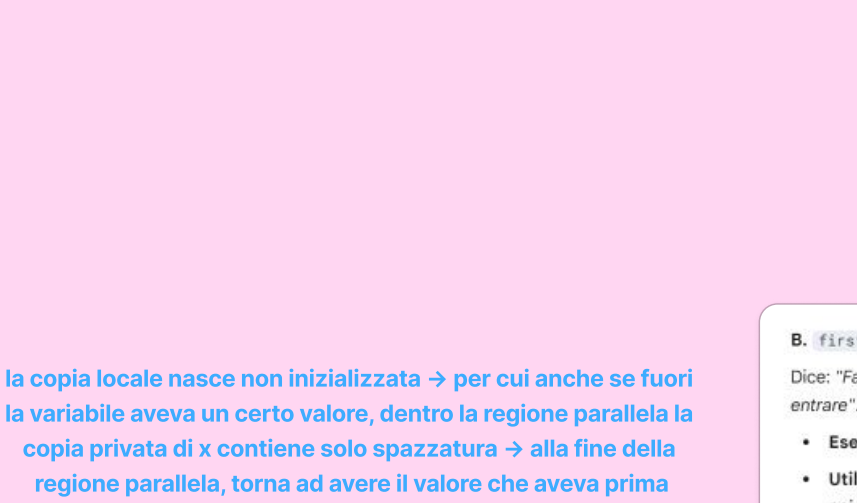
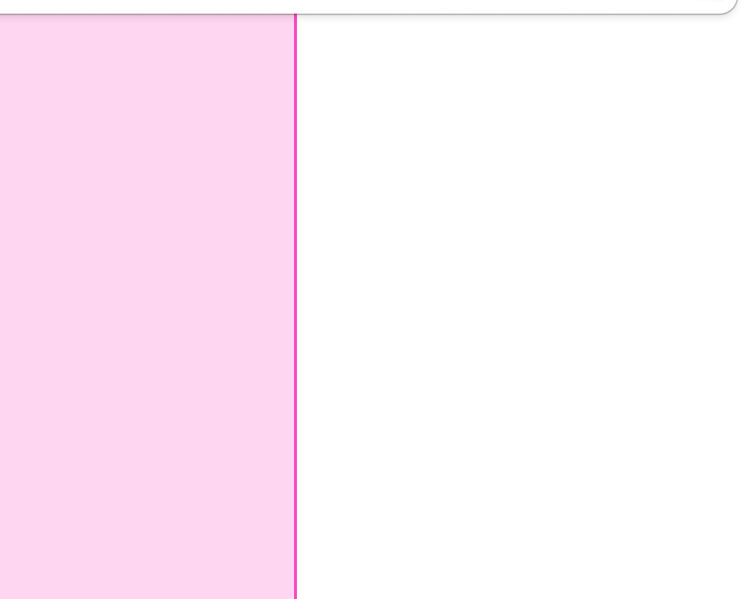


Tabella riassuntiva "Cosa usare quando?"

Situazione	Direttiva Consigliata	Perché?
Devo aggiornare un contatore (`i++`)	`atomic`	Più veloce, hardware-based.
Devo aggiornare una struttura dati complessa	`critical`	`atomic` non supporta blocchi di codice.
Devo fare I/O o setup una tantum	`master` o `single`	`master` se non vuoi attese, `single` se vuoi sicurezza (barriera).
Devo assicurarmi che tutti abbiano finito prima di proseguire	`barrier`	Sincronizzazione globale esplicita.
Devo stampare i risultati nell'ordine corretto	`ordered`	Forza l'ordine sequenziale nel loop.

- **data environment**
  - because OpenMP is a shared-memory model, variables are **shared by default**
    - convenient but dangerous
  - you must explicitly define the "Scope" of your variables to avoid data races
    - they are used in conjunction with several directives to
      - define how and which data variables in the serial section of the program are transferred to the parallel sections
      - define which variables will be visible to all threads in the parallel sections and which variables will be privately allocated to all threads
    - we can have
      - **private variables**
        - `#pragma omp <name> private (list)`
          - we are telling the compiler that the variables in the list are local to each thread
        - **warning**
          - a private variable creates a *new* object for each thread and it is uninitialized
            - la copia locale nasce non inizializzata → per cui anche se fuori la variabile aveva un certo valore, dentro la regione parallela la copia privata di x contiene solo spazzatura → alla fine della regione parallela, torna ad avere il valore che aveva prima
          - even if the variable x had a value of 10 before the parallel region, inside the region, the private x starts with a garbage value (unless you use firstprivate)
      - **shared variables**
        - `#pragma omp <name> shared (list)`
          - declares variables in its list to be shared among all threads in the team
        - a shared variable exists in only one memory location
          - all threads can read or write to that address
          - it is the programmer's repsonsability to ensure that multiple threads propertly access shared variables
      - **default clause**
        - ex. #pragma omp parallel default(none) shared (n,a) private (b)
        - `#pragma omp <name> default (shared | none)`
          - we can set the behavior for all variables
          - default(shared) is already the default, so it can be omitted
        - **best practice**
          - it is highly recommended to use default(none)
            - this forces you to explicitly list the scope (private or shared) for *every* variable used in the block
            - it is tedious but it prevents accidental shared access bugs

  le regole di default per le variabili sono:
  - le variabili dichiarate FUORI dalla regione parallela sono SHARED per default
  - le variabili dichiarate DENTRO la regione parallela sono PRIVATE
  - gli indici dei loop nei costrutti omp for sono automaticamente PRIVATE



B. `firstprivate(x)` **(Input)**

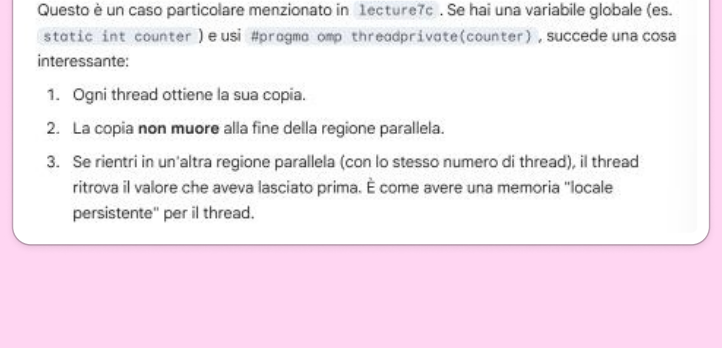
Dice: "*Fai una copia privata di* `x`*, ma inizializzala col valore che* `x` *aveva appena prima di entrare*".

- **Esempio:** Se `x = 10` fuori, ogni thread parte con la sua copia uguale a 10.
- **Utilità:** Fondamentale quando i thread devono leggere un valore iniziale comune ma poi modificarlo per conto loro senza disturbare gli altri.

C. `lastprivate(x)` **(Output)**

Dice: "*Copia fuori il valore finale*".

- **Il problema:** Con `private`, le modifiche vengono perse. Con `lastprivate`, OpenMP prende il valore dal thread che ha eseguito **l'ultima iterazione logica** del ciclo (es. `i == N-1`) e lo copia nella variabile originale esterna.
- **Nota:** Non conta chi finisce per ultimo nel tempo (chi è più lento), ma chi ha fatto l'ultimo pezzo di lavoro logico (l'ultimo indice del `for`).



5. `threadprivate` **(Variabili Globali Persistenti)**

Questo è un caso particolare menzionato in `lecture7c`. Se hai una variabile globale (es. `static int counter`) e usi `#pragma omp threadprivate(counter)`, succede una cosa interessante:

1. Ogni thread ottiene la sua copia.
2. La copia **non muore** alla fine della regione parallela.
3. Se rientri in un'altra regione parallela (con lo stesso numero di thread), il thread ritrova il valore che aveva lasciato prima. È come avere una memoria "locale persistente" per il thread.

- **runtime functions**
  - OpenMP allows you to interact with the execution environment through library calls and environment variables

    modificare il sistema e modificare il comportamento mentre il programma sta girando

    - runtime library functions as omp.h are needed to know context inside your code
      - **omp_get_thread_num()** returns the ID of the current thread (0 to N-1)
        - si usa se si vuole dividere il lavoro manualmente senza usare omp for
      - **omp_get_num_threads()** returns the total number of threads in the current team
        - se si chiama fuori dalla regione parallela restituisce 1
      - **omp_get_wtime()** a portable "wall clock" timer, useful for measuring performance
        - restituisce il tempo trascorso in secondi da un momento fisso nel passato
    - with **environment variables** we can control the program without recompiling by setting variables in your shell
      - OMP_NUM_THREADS sets how many threads to use
      - OMP_SCHEDULE defines the scheduling policy (static, dynamic) for loops that use schedule(runtime)